Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Lenguaje de Programación II

Guía Práctica — Sesión 10

Seguridad backend: usuarios, BCrypt, autenticación JWT, roles, permisos y protección de endpoints

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	10 · 15 de octubre de 2026
Modalidad	Individual, en laboratorio	Duración	2 horas prácticas
Tecnologías	Spring Boot 4 · Spring Security 7 · JJWT 0.12 · Oracle	Proyecto base	PharmaBackend (rama master)
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2

1. Situación de la sesión

PharmaSoft ya registra ventas, administra el catálogo y emite reportes. Pero la API no pregunta quién llama: cualquiera que conozca la dirección puede borrar un producto, registrar una venta a nombre de la botica o descargar el reporte de ingresos. Además, la botica quiere separar responsabilidades: el **vendedor** atiende ventas y clientes, el **almacenero** mantiene el catálogo y solo el **administrador** ve los reportes y crea usuarios.

Hoy PharmaBackend aprenderá a **identificar** a cada usuario (autenticación) y a **decidir** qué puede hacer (autorización). Las contraseñas se guardarán como hash BCrypt, el login entregará un token JWT firmado y cada endpoint exigirá el permiso que le corresponde.

¿Cómo proteges una API REST para que solo los usuarios autenticados, y con el permiso adecuado, puedan ejecutar cada operación?

Reto 01 (actividad práctica, en clase): implementar usuarios, hash de contraseñas, login JWT y permisos por rol en PharmaBackend, y probar los accesos permitidos y denegados.

Reto 02 (actividad autónoma): construir la matriz de pruebas de seguridad y agregar el cambio de contraseña del usuario autenticado.

1.1 Resultado de aprendizaje y criterio que se trabaja

Elemento	Descripción
Resultado de aprendizaje de la unidad	Construye una SPA empresarial conectada al backend REST, con layout y navegación, CRUD de tablas independientes y dependientes, formularios cabecera–detalle, consultas y reportes; integra seguridad mediante JWT en el backend y gestión de sesión, guards e interceptores en el frontend.
Criterio 4 del producto	Protege el backend con usuarios, hash de contraseñas, autenticación JWT, roles, permisos y endpoints protegidos.

1.2 Punto de partida

Requisito	Cómo verificarlo
PharmaBackend en `master`, con Flyway y CORS (sesión 7).	`git log --oneline -1` muestra «Merge feature/consultas-reportes…».
Oracle en ejecución con el esquema de PharmaBackend.	La API arranca y Swagger abre en `http://localhost:8080/swagger-ui.html`.
Postman instalado (opcional) o Swagger.	Puedes enviar un `GET /api/v1/productos`.

1.3 Ideas clave antes de programar

Concepto	Qué significa en PharmaBackend
Autenticación	¿Quién eres? El usuario envía usuario y contraseña a `POST /api/v1/auth/login` y recibe un token. Si falla: **401**.
Autorización	¿Qué puedes hacer? Cada endpoint exige un permiso. Si el usuario autenticado no lo tiene: **403**.
Hash BCrypt	Huella irreversible de la contraseña, con sal aleatoria y costo configurable. Se guarda el hash, nunca la contraseña.
JWT	Token firmado con tres partes `header.payload.signature`. El payload se puede leer (Base64), pero no alterar sin romper la firma. Tiene fecha de expiración.
Stateless	El servidor no guarda sesiones: cada petición trae su token en `Authorization: Bearer <token>`.
Empleado y usuario	El empleado es la persona; el usuario, su cuenta de acceso. Relación **1 a 1**: un empleado tiene a lo sumo un usuario.
Usuario y rol	Relación **muchos a muchos** (tabla `usuario_roles`): un usuario puede tener varios roles (ADMIN, VENDEDOR, ALMACENERO) y un rol, muchos usuarios.
Permiso	Capacidad fina (VENTAS_REGISTRAR, REPORTES_VER…). Cada rol otorga un conjunto de permisos; con varios roles, el usuario recibe la unión.

Permiso	ADMIN	VENDEDOR	ALMACENERO	Endpoints que protege
CATALOGO_LEER	✓	✓	✓	GET categorías y productos
CATALOGO_ESCRIBIR	✓		✓	POST, PUT y DELETE de categorías y productos

Concepto	Qué significa en PharmaBackend			
CLIENTES_LEER	✓	✓		GET clientes
CLIENTES_ESCRIBIR	✓	✓		POST, PUT y DELETE de clientes
VENTAS_LEER	✓	✓		GET ventas y búsqueda
VENTAS_REGISTRAR	✓	✓		POST ventas
REPORTES_VER	✓			GET reportes (con @PreAuthorize)
USUARIOS_ADMINISTRAR	✓			GET y POST usuarios

2. Actividad práctica

Volvamos a la botica: el administrador quiere que mañana cada trabajador entre con su propio usuario y solo vea lo que le corresponde. Con el Reto 01 protegerás PharmaBackend paso a paso y comprobarás, con pruebas, qué responde la API a cada usuario.

Campo	Contenido
Título	Seguridad backend: usuarios, BCrypt, login JWT y autorización por roles y permisos en PharmaBackend
Objetivo	Implementa en PharmaBackend la autenticación con JWT y la autorización por permisos agrupados en roles, guardando las contraseñas con BCrypt y respondiendo 401 y 403 con el formato de error de la API, y lo comprueba con credenciales válidas e inválidas, tokens expirados o alterados y accesos permitidos y denegados.
Duración	100 minutos de trabajo + 20 minutos de pruebas y retroalimentación

Antes de empezar

Al terminar esta guía **todos los endpoints**, salvo el login, el health y Swagger, exigirán token. La SPA de las sesiones 7 a 9 recibirá 401 hasta la próxima sesión, en la que agregarás el login y el interceptor en Angular. Es el comportamiento esperado.

Paso 1. Crear la rama de trabajo

```
cd PharmaBackend
git checkout master
git pull
git checkout -b feature/seguridad-jwt
```

Paso 2. Agregar las dependencias

`spring-boot-starter-security` trae Spring Security y BCrypt. JJWT genera y valida los tokens: `jjwt-api` es la interfaz que usa tu código, y `jjwt-impl` y `jjwt-jackson` se necesitan solo al ejecutar. Agrega la propiedad de versión en `<properties>` y las dependencias en `<dependencies>`:

Archivo: `pom.xml (fragmentos)`

```
<properties>
    <java.version>21</java.version>
    <jjwt.version>0.12.6</jjwt.version>
</properties>

<!-- Seguridad: autenticacion, autorizacion y BCrypt -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <!-- JWT: generar y validar tokens firmados (HS256/384/512 segun la clave) -->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>${jjwt.version}</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>${jjwt.version}</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>${jjwt.version}</version>
            <scope>runtime</scope>
        </dependency>

<!-- en las dependencias de prueba -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security-test</artifactId>
    <scope>test</scope>
</dependency>
```

Recarga el proyecto Maven (en IntelliJ, el ícono «Load Maven Changes»). Si en este momento ejecutas la API, verás que todos los endpoints responden 401: Spring Security protege todo por defecto hasta que configures tus propias reglas.

Paso 3. Tablas de seguridad con Flyway

El modelo tiene cuatro tablas:

Tabla	Propósito	Relación
`empleados`	Trabajadores de la botica (DNI, nombres, cargo, correo).	1 a 1 con usuarios
`usuarios`	Cuenta de acceso: username, hash de la contraseña, estado y `empleado_id`.	La FK `empleado_id` es **UNIQUE**: eso la hace 1 a 1
`roles`	ADMIN, VENDEDOR y ALMACENERO.	Muchos a muchos con usuarios
`usuario_roles`	Tabla intermedia con la PK compuesta (`usuario_id`, `rol_id`).	Resuelve la relación muchos a muchos

Crea una migración nueva. Flyway la aplica una sola vez al arrancar y la registra en `flyway_schema_history`; **nunca modifiques V1**. La columna `password` guarda el hash BCrypt (60 caracteres). Los usuarios de prueba ya traen su hash; las contraseñas en texto plano están solo en el comentario para que puedas probar. Observa a la supervisora: tiene **dos** roles, y a la cajera María, que aún no tiene usuario.

Archivo: `src/main/resources/db/migration/V2__seguridad.sql`

```
-- ========================================================================
-- V2: seguridad
--
--   empleados 1 ── 1 usuarios      un empleado tiene a lo sumo un usuario
--   usuarios  N ── M roles         un usuario puede tener varios roles
--                                  (tabla intermedia usuario_roles)
```

```
--
-- La columna password guarda el hash BCrypt (60 caracteres), nunca la
-- contrasena en texto plano. Los permisos de cada rol se definen en el
-- backend (clase PermisosPorRol).
-- ========================================================================


create table empleados (
    id                number(19,0) generated by default as identity,
    dni               varchar2(8 char)   not null,
    nombres           varchar2(100 char) not null,
    apellidos         varchar2(100 char) not null,
    cargo             varchar2(60 char)  not null,
    email             varchar2(150 char) not null,
    telefono          varchar2(15 char),
    estado            boolean            not null,
    fecha_creacion    timestamp(6)       not null,
    fecha_modificacion timestamp(6),
    constraint pk_empleados primary key (id),
    constraint uk_empleados_dni unique (dni)
);

create unique index ux_empleados_email on empleados (lower(email));


create table roles (
    id          number(19,0) generated by default as identity,
    nombre      varchar2(30 char)  not null,
    descripcion varchar2(150 char),
    constraint pk_roles primary key (id),
    constraint uk_roles_nombre unique (nombre)
);

create table usuarios (
    id                number(19,0) generated by default as identity,
    username          varchar2(50 char)  not null,
    password          varchar2(100 char) not null,
    estado            boolean            not null,
    empleado_id       number(19,0)       not null,
    fecha_creacion    timestamp(6)       not null,
    fecha_modificacion timestamp(6),
    constraint pk_usuarios primary key (id),
    -- UNIQUE en la FK: es lo que convierte la relacion en 1 a 1
    constraint uk_usuarios_empleado unique (empleado_id),
    constraint fk_usuarios_empleado
        foreign key (empleado_id) references empleados (id)
);

create unique index ux_usuarios_username on usuarios (upper(username));

-- Tabla intermedia de la relacion muchos a muchos
create table usuario_roles (
    usuario_id number(19,0) not null,
    rol_id     number(19,0) not null,
    constraint pk_usuario_roles primary key (usuario_id, rol_id),
    constraint fk_usuario_roles_usuario
        foreign key (usuario_id) references usuarios (id),
    constraint fk_usuario_roles_rol
        foreign key (rol_id) references roles (id)
);


create index ix_usuario_roles_rol on usuario_roles (rol_id);


-- ------------------------------------------------------------------------
-- Roles
-- ------------------------------------------------------------------------
insert into roles (nombre, descripcion) values ('ADMIN', 'Administra usuarios, ve reportes y opera todo el
sistema');
insert into roles (nombre, descripcion) values ('VENDEDOR', 'Atiende clientes y registra ventas');
insert into roles (nombre, descripcion) values ('ALMACENERO', 'Mantiene categorias y productos');


-- ------------------------------------------------------------------------
-- Empleados (el ultimo aun no tiene usuario: sirve para probar el alta)
-- ------------------------------------------------------------------------
insert into empleados (dni, nombres, apellidos, cargo, email, telefono, estado, fecha_creacion)
values ('40112233', 'Rosa Elena', 'Mendoza Ticona', 'Administradora', 'rmendoza@pharmasoft.pe',
'951000111', true, systimestamp);
insert into empleados (dni, nombres, apellidos, cargo, email, telefono, estado, fecha_creacion)
values ('41223344', 'Lucia', 'Ramos Cutipa', 'Vendedora', 'lramos@pharmasoft.pe', '951000222', true,
systimestamp);
insert into empleados (dni, nombres, apellidos, cargo, email, telefono, estado, fecha_creacion)
values ('42334455', 'Pedro', 'Quispe Mamani', 'Almacenero', 'pquispe@pharmasoft.pe', null, true,
systimestamp);
insert into empleados (dni, nombres, apellidos, cargo, email, telefono, estado, fecha_creacion)
```

```
values ('43445566', 'Ana', 'Torres Vilca', 'Jefa de tienda', 'atorres@pharmasoft.pe', '951000444', true,
systimestamp);
insert into empleados (dni, nombres, apellidos, cargo, email, telefono, estado, fecha_creacion)
values ('44556677', 'Jorge', 'Apaza Huanca', 'Vendedor', 'japaza@pharmasoft.pe', null, true, systimestamp);
insert into empleados (dni, nombres, apellidos, cargo, email, telefono, estado, fecha_creacion)
values ('45667788', 'Maria', 'Huaman Torres', 'Cajera', 'mhuaman@pharmasoft.pe', null, true, systimestamp);


-- ------------------------------------------------------------------------
-- Usuarios de prueba (SOLO para desarrollo y clase).
-- Contrasenas en texto plano, para probar el login:
--   admin      / Admin2026*       ADMIN                    (Rosa Mendoza)
--   vendedor   / Vendedor2026*    VENDEDOR                 (Lucia Ramos)
--   almacen    / Almacen2026*     ALMACENERO               (Pedro Quispe)
--   supervisor / Supervisor2026*  VENDEDOR + ALMACENERO    (Ana Torres)
--   inactivo   / Inactivo2026*    VENDEDOR, deshabilitado  (Jorge Apaza)
-- ------------------------------------------------------------------------
insert into usuarios (username, password, estado, empleado_id, fecha_creacion)
values ('admin', '$2a$10$NNNgOCCajIjVDGhg3u7sHOwLlLrVMdIALH/64AmPlFmmpiG1yinGa', true,
        (select id from empleados where dni = '40112233'), systimestamp);
insert into usuarios (username, password, estado, empleado_id, fecha_creacion)
values ('vendedor', '$2a$10$EXmYQXoQUBed/ZQHfPVA1.eTtSKOdgQpsz3/ZODq.za2hyGfV40We', true,
        (select id from empleados where dni = '41223344'), systimestamp);
insert into usuarios (username, password, estado, empleado_id, fecha_creacion)
values ('almacen', '$2a$10$ciG8Sz1SQTaIK5Cau0G/5u1LqIPQvcujkc5hPRaGO/Fd7ChrFBkFK', true,
        (select id from empleados where dni = '42334455'), systimestamp);
insert into usuarios (username, password, estado, empleado_id, fecha_creacion)
values ('supervisor', '$2a$10$j8xXh/XBHa/3jkViqNkMs.oZtvG6kL/QyF49kDAnnDrqktpT75sJK', true,
        (select id from empleados where dni = '43445566'), systimestamp);
insert into usuarios (username, password, estado, empleado_id, fecha_creacion)
values ('inactivo', '$2a$10$oi118hH.Nfj8jY0/MOIWJ.i34FhMDm2jy0/OpPMjSMUPh1.W/C6De', false,
        (select id from empleados where dni = '44556677'), systimestamp);


-- ------------------------------------------------------------------------
-- Roles de cada usuario (la supervisora tiene dos)
-- ------------------------------------------------------------------------
insert into usuario_roles (usuario_id, rol_id)
select u.id, r.id from usuarios u, roles r where u.username = 'admin' and r.nombre = 'ADMIN';
insert into usuario_roles (usuario_id, rol_id)
select u.id, r.id from usuarios u, roles r where u.username = 'vendedor' and r.nombre = 'VENDEDOR';
insert into usuario_roles (usuario_id, rol_id)
select u.id, r.id from usuarios u, roles r where u.username = 'almacen' and r.nombre = 'ALMACENERO';
insert into usuario_roles (usuario_id, rol_id)
select u.id, r.id from usuarios u, roles r where u.username = 'supervisor' and r.nombre in ('VENDEDOR',
'ALMACENERO');
insert into usuario_roles (usuario_id, rol_id)
select u.id, r.id from usuarios u, roles r where u.username = 'inactivo' and r.nombre = 'VENDEDOR';
```

¿De dónde salen esos hashes?

Se generaron con BCrypt (costo 10). Si ejecutas `new BCryptPasswordEncoder().encode("Admin2026*")` obtendrás un hash **distinto** cada vez, porque la sal es aleatoria; aun así, `matches("Admin2026*", hash)` devuelve `true` para cualquiera de ellos.

Paso 4. Entidades y repositorios

`Empleado` y `Rol` son entidades simples. En `Usuario` están las dos relaciones: `@OneToOne` con `@JoinColumn(unique = true)` hacia el empleado (Usuario es el lado dueño porque tiene la FK) y `@ManyToMany` con `@JoinTable` hacia los roles. Los roles se cargan **EAGER** porque en cada petición hay que conocerlos para calcular los permisos; el empleado, **LAZY**, porque solo se lee al mostrar sus datos.

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/entity/Empleado.java`

```
package pe.edu.upeu.PharmaBackend.entity;


import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
```

```
import java.time.LocalDateTime;

/*
 * Trabajador de la botica. No todos los empleados tienen usuario:
 * la relacion 1 a 1 se mapea en Usuario (lado duenio, con la FK).
 */
@Entity
@Table(name = "empleados")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Empleado {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 8)
    private String dni;

    @Column(nullable = false, length = 100)
    private String nombres;

    @Column(nullable = false, length = 100)
    private String apellidos;

    @Column(nullable = false, length = 60)
    private String cargo;

    @Column(nullable = false, length = 150)
    private String email;

    @Column(length = 15)
    private String telefono;

    @Column(nullable = false)
    private Boolean estado;

    @Column(name = "fecha_creacion", nullable = false, updatable = false)
    private LocalDateTime fechaCreacion;

    @Column(name = "fecha_modificacion")
    private LocalDateTime fechaModificacion;

    public String getNombreCompleto() {
        return nombres + " " + apellidos;
    }

    @PrePersist
    public void prePersist() {
        this.fechaCreacion = LocalDateTime.now();
        if (estado == null) {
            estado = true;
        }
    }

    @PreUpdate
    public void preUpdate() {
        this.fechaModificacion = LocalDateTime.now();
    }
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/entity/Rol.java`

```
package pe.edu.upeu.PharmaBackend.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

/*
 * Rol del sistema (ADMIN, VENDEDOR, ALMACENERO).
 * Los permisos de cada rol estan en PermisosPorRol.
 */
@Entity
@Table(name = "roles")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 30)
    private String nombre;

    @Column(length = 150)
    private String descripcion;
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/entity/Usuario.java`

```
package pe.edu.upeu.PharmaBackend.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

/*
 * Cuenta de acceso de un empleado.
 *
 *  - empleado: relacion 1 a 1. La FK empleado_id esta en esta tabla y es
 *    UNIQUE, asi un empleado no puede tener dos usuarios.
 *  - roles: relacion muchos a muchos a traves de la tabla usuario_roles.
 *    Se carga EAGER porque en cada peticion hay que conocer los roles
 *    para calcular los permisos, y son pocos.
 *
 * La columna password guarda SIEMPRE el hash BCrypt, nunca el texto plano.
 */
@Entity
@Table(name = "usuarios")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String username;

    @Column(nullable = false, length = 100)
    private String password;
```

```
    @Column(nullable = false)
    private Boolean estado;

    @OneToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "empleado_id", nullable = false, unique = true)
    private Empleado empleado;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
            name = "usuario_roles",
            joinColumns = @JoinColumn(name = "usuario_id"),
            inverseJoinColumns = @JoinColumn(name = "rol_id")
    )
    private Set<Rol> roles = new HashSet<>();

    @Column(name = "fecha_creacion", nullable = false, updatable = false)
    private LocalDateTime fechaCreacion;

    @Column(name = "fecha_modificacion")
    private LocalDateTime fechaModificacion;

    @PrePersist
    public void prePersist() {
        this.fechaCreacion = LocalDateTime.now();
        if (estado == null) {
            estado = true;
        }
    }

    @PreUpdate
    public void preUpdate() {
        this.fechaModificacion = LocalDateTime.now();
    }
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/repository/EmpleadoRepository.java`

```
package pe.edu.upeu.PharmaBackend.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import pe.edu.upeu.PharmaBackend.entity.Empleado;

public interface EmpleadoRepository extends JpaRepository<Empleado, Long> {
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/repository/RolRepository.java`

```
package pe.edu.upeu.PharmaBackend.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import pe.edu.upeu.PharmaBackend.entity.Rol;

public interface RolRepository extends JpaRepository<Rol, Long> {
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/repository/UsuarioRepository.java`

```
package pe.edu.upeu.PharmaBackend.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import pe.edu.upeu.PharmaBackend.entity.Usuario;

import java.util.Optional;
import java.util.Set;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByUsernameIgnoreCase(String username);

    boolean existsByUsernameIgnoreCase(String username);

    // Relacion 1 a 1: un empleado no puede tener dos usuarios
    boolean existsByEmpleadoId(Long empleadoId);

    // Ids de los empleados que ya tienen usuario
    @Query("select u.empleado.id from Usuario u")
    Set<Long> findEmpleadoIdsConUsuario();
}
```

Paso 5. Permisos de cada rol

Los roles viven en la tabla `roles`; los permisos que otorga cada uno se definen en el código. Spring Security trabaja con **authorities** (textos): el usuario recibirá un `ROLE_<rol>` por **cada** rol y la **unión** de los permisos de todos sus roles. La supervisora (VENDEDOR + ALMACENERO) puede vender y también editar el catálogo.

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/enums/Permiso.java`

```
package pe.edu.upeu.PharmaBackend.enums;

/*
 * Permisos finos de la aplicacion.
 *
 * Cada permiso se convierte en una "authority" de Spring Security con
 * el mismo nombre (por ejemplo, VENTAS_REGISTRAR) y se usa para
 * proteger endpoints con hasAuthority(...).
 */
public enum Permiso {
    CATALOGO_LEER,
    CATALOGO_ESCRIBIR,
    CLIENTES_LEER,
    CLIENTES_ESCRIBIR,
    VENTAS_LEER,
    VENTAS_REGISTRAR,
    REPORTES_VER,
    USUARIOS_ADMINISTRAR
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/security/PermisosPorRol.java`

```
package pe.edu.upeu.PharmaBackend.security;

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import pe.edu.upeu.PharmaBackend.entity.Rol;
import pe.edu.upeu.PharmaBackend.enums.Permiso;

import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.EnumSet;
import java.util.List;
import java.util.Map;
```

```
import java.util.Set;
import java.util.TreeSet;


/*
 * Permisos que otorga cada rol.
 *
 * Los roles estan en la tabla roles; los permisos, aqui. Si un usuario
 * tiene varios roles (relacion muchos a muchos), recibe la UNION de sus
 * permisos. Un rol que no figura en el mapa no otorga ningun permiso.
 */
public final class PermisosPorRol {

    private static final Map<String, Set<Permiso>> PERMISOS = Map.of(
            "ADMIN", Collections.unmodifiableSet(EnumSet.allOf(Permiso.class)),
            "VENDEDOR", Collections.unmodifiableSet(EnumSet.of(
                    Permiso.CATALOGO_LEER,
                    Permiso.CLIENTES_LEER,
                    Permiso.CLIENTES_ESCRIBIR,
                    Permiso.VENTAS_LEER,
                    Permiso.VENTAS_REGISTRAR)),
            "ALMACENERO", Collections.unmodifiableSet(EnumSet.of(
                    Permiso.CATALOGO_LEER,
                    Permiso.CATALOGO_ESCRIBIR))
    );

    private PermisosPorRol() {
    }

    public static Set<Permiso> de(String nombreRol) {
        return PERMISOS.getOrDefault(nombreRol, Set.of());
    }

    /* Union de los permisos de todos los roles, ordenada alfabeticamente. */
    public static List<String> permisos(Collection<Rol> roles) {
        Set<String> nombres = new TreeSet<>();
        roles.forEach(rol -> de(rol.getNombre())
                .forEach(permiso -> nombres.add(permiso.name())));
        return new ArrayList<>(nombres);
    }

    /*
     * Authorities para Spring Security: un "ROLE_<rol>" por cada rol (para
     * hasRole) mas un authority por cada permiso (para hasAuthority).
     */
    public static List<GrantedAuthority> authorities(Collection<Rol> roles) {
        List<GrantedAuthority> authorities = new ArrayList<>();
        roles.stream()
                .map(Rol::getNombre)
                .sorted()
                .forEach(nombre -> authorities.add(new SimpleGrantedAuthority("ROLE_" + nombre)));
        permisos(roles).forEach(permiso ->
                authorities.add(new SimpleGrantedAuthority(permiso)));
        return authorities;
    }
}
```

Paso 6. Configurar y generar el JWT

El secreto firma los tokens: con él se puede fabricar un token válido para cualquier usuario, así que nunca se publica. Debe ser Base64 y tener **al menos 256 bits** (32 bytes). En desarrollo va en `application-dev.yaml`; en producción llega por variable de entorno. Agrega al final de cada archivo, dentro de `app:`:

Archivo: `src/main/resources/application-dev.yaml (al final)`

```
# JWT en desarrollo. El secreto es una clave Base64 de 48 bytes (384 bits).
# Es SOLO para clase: en produccion llega por variable de entorno.
  jwt:
    secret: w9Z6dP/dzykNH/uiI3Z/KGjBlP0H4Ji+Tuq4fm2cTNuLD4jWafu1gBL74V2SJEvy
    expiracion-minutos: 60
```

Archivo: `src/main/resources/application-prod.yaml (al final)`

```
# En produccion el secreto NUNCA se escribe en el repositorio:
# llega por variable de entorno (minimo 32 bytes en Base64).
  jwt:
    secret: ${JWT_SECRET}
    expiracion-minutos: ${JWT_EXPIRACION_MINUTOS:30}
```

Para generar tu propio secreto puedes usar `openssl rand -base64 48` o, en PowerShell, `[Convert]::ToBase64String((1..48 | % { Get-Random -Max 256 }))`.

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/security/JwtService.java`

```
package pe.edu.upeu.PharmaBackend.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.Date;
import java.util.List;

/*
 * Genera y valida los JWT de la aplicacion.
 *
 * El token se firma con HMAC-SHA (HS256/HS384/HS512 segun el largo de
 * la clave) usando un secreto que solo conoce el servidor. Cualquier
 * cambio en el token invalida la firma.
 *
 * El payload NO esta cifrado: cualquiera puede leerlo decodificando
 * Base64. Por eso nunca se guardan contrasenas ni datos sensibles en el.
 */
@Service
public class JwtService {

    private final SecretKey clave;
    private final long expiracionMinutos;

    public JwtService(
            @Value("${app.jwt.secret}") String secretoBase64,
            @Value("${app.jwt.expiracion-minutos:60}") long expiracionMinutos) {

        // La clave debe tener al menos 256 bits (32 bytes); si es mas
        // corta, Keys.hmacShaKeyFor lanza WeakKeyException al arrancar.
        this.clave = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretoBase64));
        this.expiracionMinutos = expiracionMinutos;
    }

    /*
     * Token con: sub = username, authorities = rol y permisos,
     * iat = emitido en, exp = expira en.
     */
    public String generarToken(UserDetails usuario) {

        Instant ahora = Instant.now();
        Instant expira = ahora.plus(expiracionMinutos, ChronoUnit.MINUTES);

        List<String> authorities = usuario.getAuthorities()
                .stream()
                .map(GrantedAuthority::getAuthority)
                .toList();

        return Jwts.builder()
                .subject(usuario.getUsername())
                .claim("authorities", authorities)
                .issuedAt(Date.from(ahora))
                .expiration(Date.from(expira))
                .signWith(clave)
                .compact();
    }
```

```
    /*
     * Valida firma y expiracion y devuelve el username (claim "sub").
     * Lanza ExpiredJwtException si expiro y JwtException si la firma
     * o el formato no son validos.
     */
    public String extraerUsername(String token) {
        return leerClaims(token).getSubject();
    }

    public Claims leerClaims(String token) {
        return Jwts.parser()
                .verifyWith(clave)
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }

    public long getExpiracionSegundos() {
        return expiracionMinutos * 60;
    }
}
```

Paso 7. Cargar el usuario desde la base de datos

Spring Security no sabe nada de tu tabla `usuarios`. `UsuarioDetailsService` es el puente: dado un username, devuelve el hash, las authorities y si la cuenta está habilitada.

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/security/UsuarioDetailsService.java`

```
package pe.edu.upeu.PharmaBackend.security;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.edu.upeu.PharmaBackend.entity.Usuario;
import pe.edu.upeu.PharmaBackend.repository.UsuarioRepository;

/*
 * Puente entre la tabla usuarios y Spring Security.
 *
 * Spring Security pide los datos del usuario por su username y recibe
 * un UserDetails con: el hash de la contrasena, las authorities (todos
 * sus roles y la union de sus permisos) y si la cuenta esta habilitada.
 */
@Service
public class UsuarioDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {

        Usuario usuario = usuarioRepository
                .findByUsernameIgnoreCase(username.trim())
                .orElseThrow(() -> new UsernameNotFoundException(
                        "Usuario no encontrado: " + username));

        return User.withUsername(usuario.getUsername())
                .password(usuario.getPassword())
                .authorities(PermisosPorRol.authorities(usuario.getRoles()))
                // Deshabilitado si la cuenta o el empleado estan inactivos
                .disabled(!(Boolean.TRUE.equals(usuario.getEstado())
```

```
                        && Boolean.TRUE.equals(usuario.getEmpleado().getEstado())))
                .build();
    }
}
```

Paso 8. Filtro que valida el token en cada petición

El filtro se ejecuta antes de la autorización. Si el token es válido deja al usuario autenticado en el `SecurityContext`; si no lo es, guarda el motivo para responder un 401 claro. Observa que **no** lleva `@Component`: lo crea `SecurityConfig` para que solo actúe dentro de la cadena de seguridad.

Archivo:
`src/main/java/pe/edu/upeu/PharmaBackend/security/JwtAuthenticationFilter.java`

```
package pe.edu.upeu.PharmaBackend.security;


import io.jsonwebtoken.ExpiredJwtException;
import io.jsonwebtoken.JwtException;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpHeaders;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.web.filter.OncePerRequestFilter;


import java.io.IOException;


/*
 * Se ejecuta una vez por peticion, antes de la autorizacion.
 *
 * 1. Lee el encabezado  Authorization: Bearer <token>.
 * 2. Valida firma y expiracion del token.
 * 3. Carga el usuario y, si esta habilitado, lo deja autenticado en el
 *    SecurityContext para el resto de la peticion.
 *
 * Si no hay token o es invalido, NO corta la peticion: la deja seguir
 * sin autenticar. Si el endpoint es protegido, Spring Security llamara
 * al AuthenticationEntryPoint, que responde 401 con el motivo guardado
 * en el atributo ATRIBUTO_ERROR.
 *
 * No se registra como @Component: lo crea SecurityConfig para que solo
 * actue dentro de la cadena de seguridad.
 */
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    public static final String ATRIBUTO_ERROR = "jwt.error";

    private static final Logger log =
            LoggerFactory.getLogger(JwtAuthenticationFilter.class);

    private static final String PREFIJO = "Bearer ";

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(
            JwtService jwtService,
            UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
```

```
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        String encabezado = request.getHeader(HttpHeaders.AUTHORIZATION);

        if (encabezado == null || !encabezado.startsWith(PREFIJO)) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = encabezado.substring(PREFIJO.length()).trim();

        try {
            String username = jwtService.extraerUsername(token);

            if (username != null
                    && SecurityContextHolder.getContext().getAuthentication() == null) {

                // Se consulta la base en cada peticion: si el usuario fue
                // deshabilitado despues de emitir el token, deja de entrar.
                UserDetails usuario = userDetailsService.loadUserByUsername(username);

                if (usuario.isEnabled()) {
                    UsernamePasswordAuthenticationToken autenticacion =
                            new UsernamePasswordAuthenticationToken(
                                    usuario, null, usuario.getAuthorities());
                    autenticacion.setDetails(
                            new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(autenticacion);
                } else {
                    request.setAttribute(ATRIBUTO_ERROR, "El usuario está deshabilitado");
                }
            }

        } catch (ExpiredJwtException ex) {
            log.debug("Token expirado en {}", request.getRequestURI());
            request.setAttribute(ATRIBUTO_ERROR,
                    "El token ha expirado. Inicie sesión nuevamente");

        } catch (UsernameNotFoundException ex) {
            log.warn("Token de un usuario que ya no existe en {}", request.getRequestURI());
            request.setAttribute(ATRIBUTO_ERROR, "El usuario del token no existe");

        } catch (JwtException | IllegalArgumentException ex) {
            log.warn("Token inválido en {}: {}", request.getRequestURI(), ex.getMessage());
            request.setAttribute(ATRIBUTO_ERROR, "El token no es válido");
        }

        filterChain.doFilter(request, response);
    }
}
```

Paso 9. Respuestas 401 y 403 con el formato de la API

Los rechazos de seguridad ocurren en los filtros, **antes** de llegar a los controladores, por eso `@RestControllerAdvice` no los atrapa. Estas tres clases escriben el mismo JSON que `ErrorResponseDTO`.

Archivo:
`src/main/java/pe/edu/upeu/PharmaBackend/security/RespuestaErrorSeguridad.java`

```
package pe.edu.upeu.PharmaBackend.security;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.MediaType;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;

/*
```

```
 * Escribe los errores 401 y 403 con el mismo formato que
 * ErrorResponseDTO del GlobalExceptionHandler.
 *
 * Hace falta porque estos errores ocurren en los filtros de seguridad,
 * ANTES de llegar a los controladores, y alli @RestControllerAdvice no
 * interviene.
 */
final class RespuestaErrorSeguridad {

    private RespuestaErrorSeguridad() {
    }

    static void escribir(
            HttpServletRequest request,
            HttpServletResponse response,
            int status,
            String error,
            String mensaje) throws IOException {

        String json = "{"
                + "\"timestamp\":\"" + LocalDateTime.now() + "\","
                + "\"status\":" + status + ","
                + "\"error\":\"" + escapar(error) + "\","
                + "\"message\":\"" + escapar(mensaje) + "\","
                + "\"path\":\"" + escapar(request.getRequestURI()) + "\","
                + "\"validationErrors\":null"
                + "}";

        response.setStatus(status);
        response.setCharacterEncoding(StandardCharsets.UTF_8.name());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.getWriter().write(json);
    }

    private static String escapar(String texto) {
        if (texto == null) {
            return "";
        }
        return texto.replace("\\", "\\\\").replace("\"", "\\\"");
    }
}
```

Archivo:
`src/main/java/pe/edu/upeu/PharmaBackend/security/RestAuthenticationEntryPoint.java`

```
package pe.edu.upeu.PharmaBackend.security;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

import java.io.IOException;

/*
 * 401 Unauthorized: la peticion llego a un endpoint protegido SIN un
 * usuario autenticado (no envio token, el token expiro o no es valido).
 */
@Component
public class RestAuthenticationEntryPoint implements AuthenticationEntryPoint {

    @Override
    public void commence(
            HttpServletRequest request,
            HttpServletResponse response,
            AuthenticationException authException) throws IOException {

        Object motivo = request.getAttribute(JwtAuthenticationFilter.ATRIBUTO_ERROR);
        String mensaje = motivo != null
                ? motivo.toString()
                : "Debe iniciar sesión para acceder a este recurso";

        response.setHeader(HttpHeaders.WWW_AUTHENTICATE, "Bearer");
```

```
        RespuestaErrorSeguridad.escribir(
                request,
                response,
                HttpStatus.UNAUTHORIZED.value(),
                "Unauthorized",
                mensaje);
    }
}
```

Archivo:
`src/main/java/pe/edu/upeu/PharmaBackend/security/RestAccessDeniedHandler.java`

```
package pe.edu.upeu.PharmaBackend.security;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;

import java.io.IOException;

/*
 * 403 Forbidden: el usuario SI esta autenticado (token valido), pero
 * su rol no tiene el permiso que exige el endpoint.
 */
@Component
public class RestAccessDeniedHandler implements AccessDeniedHandler {

    private static final Logger log =
            LoggerFactory.getLogger(RestAccessDeniedHandler.class);

    @Override
    public void handle(
            HttpServletRequest request,
            HttpServletResponse response,
            AccessDeniedException accessDeniedException) throws IOException {

        log.warn("Acceso denegado: {} {} usuario={}",
                request.getMethod(),
                request.getRequestURI(),
                request.getUserPrincipal() != null
                        ? request.getUserPrincipal().getName()
                        : "anónimo");

        RespuestaErrorSeguridad.escribir(
                request,
                response,
                HttpStatus.FORBIDDEN.value(),
                "Forbidden",
                "No tiene permisos para realizar esta operación");
    }
}
```

Además, agrega dos manejadores en `GlobalExceptionHandler`, antes del manejador genérico. El primero responde 401 cuando el login falla; el segundo responde 403 cuando `@PreAuthorize` rechaza una operación dentro de un controlador (sin él, el manejador genérico respondería 500).

Archivo:
`src/main/java/pe/edu/upeu/PharmaBackend/exception/GlobalExceptionHandler.java (fragmento)`

```
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.core.AuthenticationException;
```

```
    /*
     * Login fallido: usuario inexistente, contraseña incorrecta o
     * usuario deshabilitado. Se responde SIEMPRE el mismo mensaje para
     * no revelar cuál de los datos falló.
     * HTTP 401
     */
    @ExceptionHandler(AuthenticationException.class)
    public ResponseEntity<ErrorResponseDTO> handleAuthentication(
            AuthenticationException ex,
            HttpServletRequest request) {

        log.warn("Autenticación fallida en {} | motivo={}",
                request.getRequestURI(),
                ex.getClass().getSimpleName());

        ErrorResponseDTO error = new ErrorResponseDTO(
                LocalDateTime.now(),
                HttpStatus.UNAUTHORIZED.value(),
                "Unauthorized",
                "Usuario o contraseña incorrectos",
                request.getRequestURI(),
                null
        );

        return ResponseEntity
                .status(HttpStatus.UNAUTHORIZED)
                .body(error);
    }



    /*
     * @PreAuthorize rechazó la operación dentro del controlador
     * (el usuario está autenticado, pero no tiene el permiso).
     * Sin este manejador caería en el genérico y respondería 500.
     * HTTP 403
     */
    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponseDTO> handleAccessDenied(
            AccessDeniedException ex,
            HttpServletRequest request) {

        log.warn("Acceso denegado en {}", request.getRequestURI());

        ErrorResponseDTO error = new ErrorResponseDTO(
                LocalDateTime.now(),
                HttpStatus.FORBIDDEN.value(),
                "Forbidden",
                "No tiene permisos para realizar esta operación",
                request.getRequestURI(),
                null
        );

        return ResponseEntity
                .status(HttpStatus.FORBIDDEN)
                .body(error);
    }
```

Paso 10. La cadena de seguridad

`SecurityConfig` reúne todo: API sin estado, CSRF desactivado (no hay cookies de sesión), CORS, reglas por URL y método, manejadores de 401 y 403 y el filtro JWT. **El orden de las reglas importa**: gana la primera que coincide, por eso las reglas `GET` van antes que la regla general de cada recurso.

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/config/SecurityConfig.java`

```
package pe.edu.upeu.PharmaBackend.config;


import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfigurationSource;
import pe.edu.upeu.PharmaBackend.security.JwtAuthenticationFilter;
import pe.edu.upeu.PharmaBackend.security.JwtService;
import pe.edu.upeu.PharmaBackend.security.RestAccessDeniedHandler;
import pe.edu.upeu.PharmaBackend.security.RestAuthenticationEntryPoint;


import static pe.edu.upeu.PharmaBackend.enums.Permiso.CATALOGO_ESCRIBIR;
import static pe.edu.upeu.PharmaBackend.enums.Permiso.CATALOGO_LEER;
import static pe.edu.upeu.PharmaBackend.enums.Permiso.CLIENTES_ESCRIBIR;
import static pe.edu.upeu.PharmaBackend.enums.Permiso.CLIENTES_LEER;
import static pe.edu.upeu.PharmaBackend.enums.Permiso.USUARIOS_ADMINISTRAR;
import static pe.edu.upeu.PharmaBackend.enums.Permiso.VENTAS_LEER;
import static pe.edu.upeu.PharmaBackend.enums.Permiso.VENTAS_REGISTRAR;


/*
 * Cadena de seguridad de la API.
 *
 *  - Sin estado (STATELESS): no hay sesion en el servidor; cada peticion
 *    trae su JWT en el encabezado Authorization.
 *  - CSRF desactivado: no se usan cookies de sesion, el token viaja en
 *    un encabezado que el navegador no envia solo.
 *  - Reglas por URL y metodo HTTP: el orden importa, gana la primera
 *    regla que coincide.
 *  - @EnableMethodSecurity habilita @PreAuthorize en los controladores
 *    (se usa en ReporteController).
 */
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {


    private static final String[] RUTAS_PUBLICAS = {
            "/api/v1/auth/login",
            "/api/health",
            "/swagger-ui.html",
            "/swagger-ui/**",
            "/v3/api-docs/**",
            "/error"
    };

    @Bean
    public SecurityFilterChain filterChain(
            HttpSecurity http,
            JwtService jwtService,
            UserDetailsService userDetailsService,
            CorsConfigurationSource corsConfigurationSource,
            RestAuthenticationEntryPoint authenticationEntryPoint,
            RestAccessDeniedHandler accessDeniedHandler) throws Exception {


        JwtAuthenticationFilter jwtFilter =
                new JwtAuthenticationFilter(jwtService, userDetailsService);


        http
                .csrf(AbstractHttpConfigurer::disable)
                .logout(AbstractHttpConfigurer::disable)
                .cors(cors -> cors.configurationSource(corsConfigurationSource))
                .sessionManagement(sesion ->
                        sesion.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(auth -> auth
                        // Publicas
                        .requestMatchers(RUTAS_PUBLICAS).permitAll()
                        .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()


                        // Catalogo: todos leen; solo ADMIN y ALMACENERO escriben
                        .requestMatchers(HttpMethod.GET,
                                "/api/v1/categorias/**", "/api/v1/productos/**")
                        .hasAuthority(CATALOGO_LEER.name())
                        .requestMatchers(
                                "/api/v1/categorias/**", "/api/v1/productos/**")
                        .hasAuthority(CATALOGO_ESCRIBIR.name())


                        // Clientes
                        .requestMatchers(HttpMethod.GET, "/api/v1/clientes/**")
                        .hasAuthority(CLIENTES_LEER.name())
                        .requestMatchers("/api/v1/clientes/**")
```

```
                        .hasAuthority(CLIENTES_ESCRIBIR.name())

                        // Ventas
                        .requestMatchers(HttpMethod.POST, "/api/v1/ventas")
                        .hasAuthority(VENTAS_REGISTRAR.name())
                        .requestMatchers(HttpMethod.GET, "/api/v1/ventas/**")
                        .hasAuthority(VENTAS_LEER.name())
                        .requestMatchers("/api/v1/ventas/**")
                        .hasAuthority(VENTAS_REGISTRAR.name())

                        // Administracion de la seguridad
                        .requestMatchers("/api/v1/usuarios/**",
                                "/api/v1/empleados/**", "/api/v1/roles/**")
                        .hasAuthority(USUARIOS_ADMINISTRAR.name())

                        // Todo lo demas (auth/me, reportes...) exige un usuario
                        // autenticado; los reportes ademas usan @PreAuthorize.
                        .anyRequest().authenticated())
                .exceptionHandling(errores -> errores
                        .authenticationEntryPoint(authenticationEntryPoint)
                        .accessDeniedHandler(accessDeniedHandler))
                .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    /*
     * BCrypt: hash lento y con "sal" aleatoria. La misma contrasena
     * produce un hash distinto cada vez y no se puede revertir.
     */
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    /*
     * Spring arma el AuthenticationManager con el UserDetailsService
     * (UsuarioDetailsService) y el PasswordEncoder declarados.
     */
    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration configuration) throws Exception {
        return configuration.getAuthenticationManager();
    }
}
```

Con Spring Security activo, CORS debe aplicarse dentro de la cadena de seguridad; si no, la petición de verificación del navegador (`OPTIONS`) recibiría 401. Reemplaza `CorsConfig` para que exponga un `CorsConfigurationSource` (también se agrega el método `PATCH`):

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/config/CorsConfig.java`

```
package pe.edu.upeu.PharmaBackend.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

/*
 * Configuracion CORS para /api/**.
 *
 * Con Spring Security activo, CORS se aplica en la cadena de seguridad
 * (SecurityConfig usa este bean). Asi la peticion de verificacion
 * (preflight OPTIONS) del navegador se responde antes de exigir el token.
 */
@Configuration
public class CorsConfig {

    private final List<String> origenesPermitidos;
```

```
    public CorsConfig(
            @Value("${app.cors.allowed-origins:"
                    + "http://localhost:5173,"
                    + "http://127.0.0.1:5173,"
                    + "http://localhost:4200,"
                    + "http://localhost:3000}")
            String[] origenesPermitidos) {

        this.origenesPermitidos = List.of(origenesPermitidos);
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {

        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(origenesPermitidos);
        config.setAllowedMethods(List.of(
                "GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of(
                "Content-Type", "Accept", "Authorization", "Origin"));
        config.setExposedHeaders(List.of("Location"));
        config.setAllowCredentials(true);
        config.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config);
        return source;
    }
}
```

Paso 11. Login y administración de usuarios

Crea el paquete `dto/auth` con los seis DTO. Para crear un usuario se envían el `empleadoId` (relación 1 a 1) y los `rolIds` (relación muchos a muchos). La respuesta de usuario **nunca** incluye la contraseña ni su hash, y devuelve datos planos del empleado y los nombres de los roles.

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/dto/auth/LoginRequestDTO.java`

```
package pe.edu.upeu.PharmaBackend.dto.auth;

import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
public class LoginRequestDTO {

    @NotBlank(message = "El usuario es obligatorio")
    private String username;

    @NotBlank(message = "La contraseña es obligatoria")
    private String password;
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/dto/auth/LoginResponseDTO.java`

```
package pe.edu.upeu.PharmaBackend.dto.auth;


import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;


import java.util.List;


/*
 * Respuesta del login. El frontend guarda el token y lo envia en cada
 * peticion como:  Authorization: Bearer <token>
 */
@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
public class LoginResponseDTO {
    private String token;
    private String tipo;
    private long expiraEnSegundos;
    private String username;
    private String nombreCompleto;
    private List<String> roles;
    private List<String> permisos;
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/dto/auth/UsuarioRequestDTO.java`

```
package pe.edu.upeu.PharmaBackend.dto.auth;


import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;


import java.util.Set;


@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
public class UsuarioRequestDTO {


    @NotBlank(message = "El usuario es obligatorio")
    @Size(min = 4, max = 50, message = "El usuario debe tener entre 4 y 50 caracteres")
    @Pattern(
            regexp = "^[a-zA-Z0-9._-]+$",
            message = "El usuario solo admite letras, números, punto, guion y guion bajo"
    )
    private String username;


    /*
     * BCrypt solo considera los primeros 72 bytes de la contrasena:
     * por eso el maximo es 72.
     */
    @NotBlank(message = "La contraseña es obligatoria")
    @Size(min = 8, max = 72, message = "La contraseña debe tener entre 8 y 72 caracteres")
    @Pattern(
            regexp = "^(?=.*[A-Za-z])(?=.*\\d).+$",
            message = "La contraseña debe contener al menos una letra y un número"
    )
    private String password;


    // Relacion 1 a 1: el empleado dueño de la cuenta
    @NotNull(message = "El empleado es obligatorio")
    private Long empleadoId;


    // Relacion muchos a muchos: uno o varios roles
    @NotEmpty(message = "Debe asignar al menos un rol")
    private Set<@NotNull(message = "El id de rol no puede ser nulo") Long> rolIds;
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/dto/auth/UsuarioResponseDTO.java`

```
package pe.edu.upeu.PharmaBackend.dto.auth;


import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;


import java.time.LocalDateTime;
import java.util.List;


/*
 * Datos publicos del usuario: NUNCA incluye la contrasena ni su hash.
 * Del empleado y de los roles se devuelven datos planos (sin anidar
 * entidades).
 */
@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
public class UsuarioResponseDTO {
    private Long id;
    private String username;
    private Long empleadoId;
    private String empleadoNombre;
    private String cargo;
    private List<String> roles;
    private List<String> permisos;
    private Boolean estado;
    private LocalDateTime fechaCreacion;
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/dto/auth/EmpleadoResponseDTO.java`

```
package pe.edu.upeu.PharmaBackend.dto.auth;


import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;


@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
public class EmpleadoResponseDTO {
    private Long id;
    private String dni;
    private String nombres;
    private String apellidos;
    private String cargo;
    private String email;
    private String telefono;
    private Boolean estado;
    private boolean tieneUsuario;
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/dto/auth/RolResponseDTO.java`

```
package pe.edu.upeu.PharmaBackend.dto.auth;


import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;


import java.util.List;


@AllArgsConstructor
@NoArgsConstructor
@Getter
@Setter
public class RolResponseDTO {
    private Long id;
    private String nombre;
    private String descripcion;
    private List<String> permisos;
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/mapper/UsuarioMapper.java`

```
package pe.edu.upeu.PharmaBackend.mapper;


import org.springframework.stereotype.Component;
import pe.edu.upeu.PharmaBackend.dto.auth.EmpleadoResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.RolResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioResponseDTO;
import pe.edu.upeu.PharmaBackend.entity.Empleado;
import pe.edu.upeu.PharmaBackend.entity.Rol;
import pe.edu.upeu.PharmaBackend.entity.Usuario;
import pe.edu.upeu.PharmaBackend.security.PermisosPorRol;


import java.util.Collection;
import java.util.List;


/*
 * Convierte las entidades de seguridad en DTO planos.
 */
@Component
public class UsuarioMapper {


    public UsuarioResponseDTO toUsuarioResponse(Usuario usuario) {
        Empleado empleado = usuario.getEmpleado();
        return new UsuarioResponseDTO(
                usuario.getId(),
                usuario.getUsername(),
                empleado.getId(),
                empleado.getNombreCompleto(),
                empleado.getCargo(),
                nombresRoles(usuario.getRoles()),
                PermisosPorRol.permisos(usuario.getRoles()),
                usuario.getEstado(),
                usuario.getFechaCreacion()
        );
    }


    public EmpleadoResponseDTO toEmpleadoResponse(Empleado empleado, boolean tieneUsuario) {
        return new EmpleadoResponseDTO(
                empleado.getId(),
                empleado.getDni(),
                empleado.getNombres(),
                empleado.getApellidos(),
                empleado.getCargo(),
                empleado.getEmail(),
                empleado.getTelefono(),
                empleado.getEstado(),
                tieneUsuario
        );
    }


    public RolResponseDTO toRolResponse(Rol rol) {
        return new RolResponseDTO(
                rol.getId(),
                rol.getNombre(),
                rol.getDescripcion(),
                PermisosPorRol.permisos(List.of(rol))
        );
```

```
    }

    public List<String> nombresRoles(Collection<Rol> roles) {
        return roles.stream()
                .map(Rol::getNombre)
                .sorted()
                .toList();
    }
}
```

El servicio de autenticación delega la verificación en `AuthenticationManager`: este busca al usuario con `UsuarioDetailsService`, revisa que esté habilitado y compara la contraseña con BCrypt. La contraseña **nunca** se escribe en el log.

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/service/service/AuthService.java`

```
package pe.edu.upeu.PharmaBackend.service.service;

import pe.edu.upeu.PharmaBackend.dto.auth.LoginRequestDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.LoginResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioResponseDTO;

public interface AuthService {

    LoginResponseDTO login(LoginRequestDTO request);

    UsuarioResponseDTO usuarioActual(String username);
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/service/impl/AuthServiceImpl.java`

```
package pe.edu.upeu.PharmaBackend.service.impl;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.edu.upeu.PharmaBackend.dto.auth.LoginRequestDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.LoginResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioResponseDTO;
import pe.edu.upeu.PharmaBackend.entity.Usuario;
import pe.edu.upeu.PharmaBackend.exception.RecursoNoEncontradoException;
import pe.edu.upeu.PharmaBackend.mapper.UsuarioMapper;
import pe.edu.upeu.PharmaBackend.repository.UsuarioRepository;
import pe.edu.upeu.PharmaBackend.security.JwtService;
import pe.edu.upeu.PharmaBackend.security.PermisosPorRol;
import pe.edu.upeu.PharmaBackend.service.service.AuthService;

import java.util.List;

@Service
public class AuthServiceImpl implements AuthService {

    private static final Logger log =
            LoggerFactory.getLogger(AuthServiceImpl.class);

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final UsuarioRepository usuarioRepository;
    private final UsuarioMapper usuarioMapper;

    public AuthServiceImpl(
            AuthenticationManager authenticationManager,
            JwtService jwtService,
            UsuarioRepository usuarioRepository,
```

```
            UsuarioMapper usuarioMapper) {
        this.authenticationManager = authenticationManager;
        this.jwtService = jwtService;
        this.usuarioRepository = usuarioRepository;
        this.usuarioMapper = usuarioMapper;
    }

    /*
     * 1. AuthenticationManager busca al usuario (UsuarioDetailsService),
     *    verifica que este habilitado y compara la contrasena con el hash
     *    usando BCrypt. Si algo falla lanza una AuthenticationException,
     *    que GlobalExceptionHandler convierte en 401.
     * 2. Si todo es correcto, se emite el JWT.
     *
     * La contrasena NUNCA se escribe en el log.
     */
    @Override
    @Transactional(readOnly = true)
    public LoginResponseDTO login(LoginRequestDTO request) {

        String username = request.getUsername().trim();

        log.info("Intento de inicio de sesión: usuario={}", username);

        Authentication autenticacion = authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(username, request.getPassword()));

        UserDetails principal = (UserDetails) autenticacion.getPrincipal();
        Usuario usuario = buscar(principal.getUsername());

        String token = jwtService.generarToken(principal);

        List<String> roles = usuarioMapper.nombresRoles(usuario.getRoles());

        log.info("Inicio de sesión correcto: usuario={} roles={}",
                usuario.getUsername(), roles);

        return new LoginResponseDTO(
                token,
                "Bearer",
                jwtService.getExpiracionSegundos(),
                usuario.getUsername(),
                usuario.getEmpleado().getNombreCompleto(),
                roles,
                PermisosPorRol.permisos(usuario.getRoles())
        );
    }

    @Override
    @Transactional(readOnly = true)
    public UsuarioResponseDTO usuarioActual(String username) {
        return usuarioMapper.toUsuarioResponse(buscar(username));
    }

    private Usuario buscar(String username) {
        return usuarioRepository.findByUsernameIgnoreCase(username)
                .orElseThrow(() -> new RecursoNoEncontradoException(
                        "Usuario no encontrado: " + username));
    }
}
```

Al crear un usuario, el servicio verifica las dos relaciones: que el empleado exista, esté activo y **aún no tenga usuario** (1 a 1), y que **todos** los roles existan (muchos a muchos).

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/service/service/UsuarioService.java`

```
package pe.edu.upeu.PharmaBackend.service.service;

import pe.edu.upeu.PharmaBackend.dto.auth.EmpleadoResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.RolResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioRequestDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioResponseDTO;

import java.util.List;

public interface UsuarioService {

    UsuarioResponseDTO crear(UsuarioRequestDTO request);

    List<UsuarioResponseDTO> listar();

    List<EmpleadoResponseDTO> listarEmpleados();

    List<RolResponseDTO> listarRoles();
}
```

Archivo:
`src/main/java/pe/edu/upeu/PharmaBackend/service/impl/UsuarioServiceImpl.java`

```
package pe.edu.upeu.PharmaBackend.service.impl;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.domain.Sort;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.edu.upeu.PharmaBackend.dto.auth.EmpleadoResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.RolResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioRequestDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioResponseDTO;
import pe.edu.upeu.PharmaBackend.entity.Empleado;
import pe.edu.upeu.PharmaBackend.entity.Rol;
import pe.edu.upeu.PharmaBackend.entity.Usuario;
import pe.edu.upeu.PharmaBackend.exception.RecursoNoEncontradoException;
import pe.edu.upeu.PharmaBackend.exception.ReglaNegocioException;
import pe.edu.upeu.PharmaBackend.mapper.UsuarioMapper;
import pe.edu.upeu.PharmaBackend.repository.EmpleadoRepository;
import pe.edu.upeu.PharmaBackend.repository.RolRepository;
import pe.edu.upeu.PharmaBackend.repository.UsuarioRepository;
import pe.edu.upeu.PharmaBackend.service.service.UsuarioService;

import java.nio.charset.StandardCharsets;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Set;
import java.util.stream.Collectors;

@Service
public class UsuarioServiceImpl implements UsuarioService {

    private static final Logger log =
            LoggerFactory.getLogger(UsuarioServiceImpl.class);

    private final UsuarioRepository usuarioRepository;
    private final EmpleadoRepository empleadoRepository;
    private final RolRepository rolRepository;
    private final PasswordEncoder passwordEncoder;
    private final UsuarioMapper usuarioMapper;

    public UsuarioServiceImpl(
            UsuarioRepository usuarioRepository,
            EmpleadoRepository empleadoRepository,
            RolRepository rolRepository,
            PasswordEncoder passwordEncoder,
            UsuarioMapper usuarioMapper) {
        this.usuarioRepository = usuarioRepository;
        this.empleadoRepository = empleadoRepository;
        this.rolRepository = rolRepository;
        this.passwordEncoder = passwordEncoder;
```

```
        this.usuarioMapper = usuarioMapper;
    }


    /*
     * Crea la cuenta de un empleado y le asigna uno o varios roles.
     *
     * Reglas:
     *  - el empleado existe (404) y esta activo (409);
     *  - el empleado aun no tiene usuario (409): relacion 1 a 1;
     *  - todos los roles existen (404): relacion muchos a muchos;
     *  - el username no se repite (409).
     */
    @Override
    @Transactional
    public UsuarioResponseDTO crear(UsuarioRequestDTO request) {

        String username = request.getUsername().trim().toLowerCase(Locale.ROOT);

        log.info("Registrando usuario={} empleadoId={} rolIds={}",
                username, request.getEmpleadoId(), request.getRolIds());

        Empleado empleado = empleadoRepository.findById(request.getEmpleadoId())
                .orElseThrow(() -> new RecursoNoEncontradoException(
                        "Empleado no encontrado con id: " + request.getEmpleadoId()));

        if (!Boolean.TRUE.equals(empleado.getEstado())) {
            throw new ReglaNegocioException(
                    "El empleado " + empleado.getNombreCompleto() + " está inactivo");
        }

        if (usuarioRepository.existsByEmpleadoId(empleado.getId())) {
            throw new ReglaNegocioException(
                    "El empleado " + empleado.getNombreCompleto() + " ya tiene un usuario");
        }

        if (usuarioRepository.existsByUsernameIgnoreCase(username)) {
            throw new ReglaNegocioException(
                    "Ya existe un usuario con el nombre: " + username);
        }

        List<Rol> roles = rolRepository.findAllById(request.getRolIds());
        if (roles.size() != request.getRolIds().size()) {
            Set<Long> encontrados = roles.stream().map(Rol::getId).collect(Collectors.toSet());
            Set<Long> faltantes = new HashSet<>(request.getRolIds());
            faltantes.removeAll(encontrados);
            throw new RecursoNoEncontradoException("Roles no encontrados: " + faltantes);
        }

        // BCrypt solo admite 72 BYTES: una ñ o una tilde ocupan 2 bytes en UTF-8
        if (request.getPassword().getBytes(StandardCharsets.UTF_8).length > 72) {
            throw new ReglaNegocioException("La contraseña no puede superar 72 bytes");
        }

        Usuario usuario = new Usuario();
        usuario.setUsername(username);
        // Se guarda el hash BCrypt, nunca la contrasena en texto plano
        usuario.setPassword(passwordEncoder.encode(request.getPassword()));
        usuario.setEstado(true);
        usuario.setEmpleado(empleado);
        usuario.setRoles(new HashSet<>(roles));

        Usuario guardado = usuarioRepository.save(usuario);

        log.info("Usuario registrado id={} usuario={} roles={}",
                guardado.getId(), username, usuarioMapper.nombresRoles(roles));

        return usuarioMapper.toUsuarioResponse(guardado);
    }

    @Override
    @Transactional(readOnly = true)
    public List<UsuarioResponseDTO> listar() {
        return usuarioRepository.findAll(Sort.by("username"))
                .stream()
                .map(usuarioMapper::toUsuarioResponse)
                .toList();
    }

    @Override
```

```
    @Transactional(readOnly = true)
    public List<EmpleadoResponseDTO> listarEmpleados() {
        Set<Long> conUsuario = usuarioRepository.findEmpleadoIdsConUsuario();

        return empleadoRepository.findAll(Sort.by("apellidos", "nombres"))
                .stream()
                .map(e -> usuarioMapper.toEmpleadoResponse(e, conUsuario.contains(e.getId())))
                .toList();
    }

    @Override
    @Transactional(readOnly = true)
    public List<RolResponseDTO> listarRoles() {
        return rolRepository.findAll(Sort.by("nombre"))
                .stream()
                .map(usuarioMapper::toRolResponse)
                .toList();
    }
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/controller/AuthController.java`

```
package pe.edu.upeu.PharmaBackend.controller;

import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.edu.upeu.PharmaBackend.dto.auth.LoginRequestDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.LoginResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioResponseDTO;
import pe.edu.upeu.PharmaBackend.service.service.AuthService;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    /*
     * Publico. Credenciales correctas -> 200 con el token.
     * Credenciales incorrectas o usuario deshabilitado -> 401.
     */
    @PostMapping("/login")
    public ResponseEntity<LoginResponseDTO> login(
            @Valid @RequestBody LoginRequestDTO request) {
        return ResponseEntity.ok(authService.login(request));
    }

    /*
     * Protegido. Devuelve el usuario dueño del token.
     * Spring inyecta la Authentication que dejo el JwtAuthenticationFilter.
     */
    @GetMapping("/me")
    public ResponseEntity<UsuarioResponseDTO> me(Authentication authentication) {
        return ResponseEntity.ok(authService.usuarioActual(authentication.getName()));
    }
}
```

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/controller/UsuarioController.java`

```
package pe.edu.upeu.PharmaBackend.controller;

import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.edu.upeu.PharmaBackend.dto.auth.EmpleadoResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.RolResponseDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioRequestDTO;
import pe.edu.upeu.PharmaBackend.dto.auth.UsuarioResponseDTO;
import pe.edu.upeu.PharmaBackend.service.service.UsuarioService;

import java.util.List;

/*
 * Administracion de la seguridad: usuarios, empleados y roles.
 * Solo usuarios con el permiso USUARIOS_ADMINISTRAR (rol ADMIN);
 * las reglas estan en SecurityConfig.
 */
@RestController
@RequestMapping("/api/v1")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping("/usuarios")
    public ResponseEntity<List<UsuarioResponseDTO>> listar() {
        return ResponseEntity.ok(usuarioService.listar());
    }

    @PostMapping("/usuarios")
    public ResponseEntity<UsuarioResponseDTO> crear(
            @Valid @RequestBody UsuarioRequestDTO request) {
        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(usuarioService.crear(request));
    }

    /* Empleados, indicando cuales ya tienen usuario (relacion 1 a 1). */
    @GetMapping("/empleados")
    public ResponseEntity<List<EmpleadoResponseDTO>> empleados() {
        return ResponseEntity.ok(usuarioService.listarEmpleados());
    }

    /* Roles disponibles con los permisos que otorga cada uno. */
    @GetMapping("/roles")
    public ResponseEntity<List<RolResponseDTO>> roles() {
        return ResponseEntity.ok(usuarioService.listarRoles());
    }
}
```

Paso 12. Autorización por método y Swagger con token

Los reportes se protegen en el propio controlador con `@PreAuthorize` (habilitado por `@EnableMethodSecurity`). Agrega la anotación sobre la clase `ReporteController`:

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/controller/ReporteController.java (fragmento)`

```
/*
 * Autorizacion a nivel de metodo: solo quien tenga el permiso
 * REPORTES_VER (rol ADMIN). Si no lo tiene, Spring lanza
 * AccessDeniedException y GlobalExceptionHandler responde 403.
 */
@RestController
@RequestMapping("/api/v1/reportes")
@PreAuthorize("hasAuthority('REPORTES_VER')")
public class ReporteController { ... }
```

Para probar desde Swagger con token, agrega esta configuración. Aparecerá el botón **Authorize**, donde se pega el token del login (sin la palabra Bearer).

Archivo: `src/main/java/pe/edu/upeu/PharmaBackend/config/OpenApiConfig.java`

```
package pe.edu.upeu.PharmaBackend.config;

import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
import io.swagger.v3.oas.annotations.info.Info;
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.security.SecurityScheme;
import org.springframework.context.annotation.Configuration;

/*
 * Agrega el boton "Authorize" en Swagger UI: se pega el token del login
 * (sin la palabra Bearer) y Swagger lo envia en cada peticion.
 */
@Configuration
@OpenAPIDefinition(
        info = @Info(title = "PharmaBackend API", version = "v1"),
        security = @SecurityRequirement(name = "bearerAuth")
)
@SecurityScheme(
        name = "bearerAuth",
        type = SecuritySchemeType.HTTP,
        scheme = "bearer",
        bearerFormat = "JWT"
)
public class OpenApiConfig {
}
```

Paso 13. Pruebas unitarias

Estas pruebas no necesitan la base de datos: verifican que el token lleve los datos correctos, que un token alterado, de otra clave o vencido sea rechazado y que cada rol tenga solo sus permisos.

Archivo: `src/test/java/pe/edu/upeu/PharmaBackend/security/JwtServiceTest.java`

```
package pe.edu.upeu.PharmaBackend.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.ExpiredJwtException;
import io.jsonwebtoken.JwtException;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import pe.edu.upeu.PharmaBackend.entity.Rol;

import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class JwtServiceTest {

    // Clave de prueba en Base64 (48 bytes = 384 bits)
    private static final String SECRETO =
            "w9Z6dP/dzykNH/uiI3Z/KGjBlP0H4Ji+Tuq4fm2cTNuLD4jWafu1gBL74V2SJEvy";

    private static final String OTRO_SECRETO =
            "q8Bx3mN0vRk7TzYp1LcW5aHs9JdFg2Ue4OiPlKjMnBvCxZaSdFgHjKlQwErTyUiO";

    private final JwtService jwtService = new JwtService(SECRETO, 60);

    private UserDetails vendedor() {
        return User.withUsername("vendedor")
                .password("no-importa")
                .authorities(PermisosPorRol.authorities(
                        List.of(new Rol(2L, "VENDEDOR", null))))
```

```
                .build();
    }

    @Test
    @DisplayName("El token generado contiene el username y las authorities")
    void generarYLeerToken() {
        String token = jwtService.generarToken(vendedor());

        Claims claims = jwtService.leerClaims(token);

        assertThat(token.split("\\.")).hasSize(3);
        assertThat(claims.getSubject()).isEqualTo("vendedor");
        assertThat(claims.getExpiration()).isAfter(claims.getIssuedAt());
        List<?> authorities = claims.get("authorities", List.class);
        assertThat(authorities)
                .contains("ROLE_VENDEDOR", "VENTAS_REGISTRAR")
                .doesNotContain("REPORTES_VER");
        assertThat(jwtService.extraerUsername(token)).isEqualTo("vendedor");
    }

    @Test
    @DisplayName("Un token alterado no pasa la verificación de firma")
    void tokenAlterado() {
        String token = jwtService.generarToken(vendedor());
        String alterado = token.substring(0, token.length() - 2)
                + (token.endsWith("AA") ? "BB" : "AA");

        assertThatThrownBy(() -> jwtService.extraerUsername(alterado))
                .isInstanceOf(JwtException.class);
    }

    @Test
    @DisplayName("Un token firmado con otra clave es rechazado")
    void tokenDeOtraClave() {
        String ajeno = new JwtService(OTRO_SECRETO, 60).generarToken(vendedor());

        assertThatThrownBy(() -> jwtService.extraerUsername(ajeno))
                .isInstanceOf(JwtException.class);
    }

    @Test
    @DisplayName("Un token expirado lanza ExpiredJwtException")
    void tokenExpirado() {
        // Expiracion negativa: el token nace vencido
        String vencido = new JwtService(SECRETO, -1).generarToken(vendedor());

        assertThatThrownBy(() -> jwtService.extraerUsername(vencido))
                .isInstanceOf(ExpiredJwtException.class);
    }

    @Test
    @DisplayName("La expiración se informa en segundos")
    void expiracionEnSegundos() {
        assertThat(jwtService.getExpiracionSegundos()).isEqualTo(3600);
    }
}
```

Archivo: `src/test/java/pe/edu/upeu/PharmaBackend/security/PermisosPorRolTest.java`

```
package pe.edu.upeu.PharmaBackend.security;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.security.core.GrantedAuthority;
import pe.edu.upeu.PharmaBackend.entity.Rol;
import pe.edu.upeu.PharmaBackend.enums.Permiso;

import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;

class PermisosPorRolTest {

    private static final Rol ADMIN = new Rol(1L, "ADMIN", null);
```

```
    private static final Rol VENDEDOR = new Rol(2L, "VENDEDOR", null);
    private static final Rol ALMACENERO = new Rol(3L, "ALMACENERO", null);


    private List<String> authorities(Rol... roles) {
        return PermisosPorRol.authorities(List.of(roles)).stream()
                .map(GrantedAuthority::getAuthority)
                .toList();
    }


    @Test
    @DisplayName("ADMIN tiene todos los permisos")
    void adminTieneTodo() {
        assertThat(PermisosPorRol.de("ADMIN")).containsExactlyInAnyOrder(Permiso.values());
        assertThat(authorities(ADMIN)).contains("ROLE_ADMIN", "REPORTES_VER", "USUARIOS_ADMINISTRAR");
    }


    @Test
    @DisplayName("VENDEDOR registra ventas, pero no ve reportes ni edita el catálogo")
    void vendedor() {
        assertThat(authorities(VENDEDOR))
                .contains("ROLE_VENDEDOR", "VENTAS_REGISTRAR", "CATALOGO_LEER")
                .doesNotContain("REPORTES_VER", "CATALOGO_ESCRIBIR", "USUARIOS_ADMINISTRAR");
    }


    @Test
    @DisplayName("Con dos roles recibe la unión de sus permisos, sin repetidos")
    void variosRoles() {
        assertThat(authorities(VENDEDOR, ALMACENERO))
                .contains("ROLE_ALMACENERO", "ROLE_VENDEDOR",
                        "CATALOGO_ESCRIBIR", "VENTAS_REGISTRAR")
                .doesNotContain("REPORTES_VER")
                .doesNotHaveDuplicates();
    }


    @Test
    @DisplayName("Un rol sin permisos definidos solo aporta ROLE_<nombre>")
    void rolDesconocido() {
        assertThat(authorities(new Rol(9L, "AUDITOR", null)))
                .containsExactly("ROLE_AUDITOR");
    }
}
```

```
./mvnw test -Dtest=JwtServiceTest,PermisosPorRolTest
# Windows: mvnw.cmd test -Dtest=JwtServiceTest,PermisosPorRolTest
```

Paso 14. Probar la seguridad de la API

Arranca la API (Flyway aplicará V2 y creará los usuarios). En Swagger, haz el login, copia el `token` de la respuesta y pégalo en **Authorize**; en Postman, usa la pestaña **Authorization → Bearer Token**. Para cambiar de usuario, vuelve a hacer login y reemplaza el token. Ejecuta las pruebas en orden y guarda una captura de cada una con el código y el cuerpo de la respuesta.

```
POST http://localhost:8080/api/v1/auth/login
Content-Type: application/json

{ "username": "vendedor", "password": "Vendedor2026*" }
```

Respuesta esperada (el token será distinto):

```
{
  "token": "eyJhbGciOiJIUzM4NCJ9.eyJzdWIiOiJ2ZW5kZWRvciIs...",
  "tipo": "Bearer",
  "expiraEnSegundos": 3600,
  "username": "vendedor",
  "nombreCompleto": "Lucia Ramos Cutipa",
  "roles": ["VENDEDOR"],
  "permisos": ["CATALOGO_LEER", "CLIENTES_ESCRIBIR", "CLIENTES_LEER", "VENTAS_LEER", "VENTAS_REGISTRAR"]
}
```

N.º	Usuario / token	Petición	Resultado esperado
1	Sin token	`GET /api/health`	200: ruta pública.
2	Sin token	`GET /api/v1/productos`	401 «Debe iniciar sesión para acceder a este recurso».
3	—	Login `admin` con contraseña incorrecta	401 «Usuario o contraseña incorrectos».
4	—	Login `inactivo` / `Inactivo2026*`	401 con el **mismo** mensaje: no revela que la cuenta existe.
5	—	Login con `username` vacío	400 con `validationErrors.username`.
6	—	Login `vendedor` / `Vendedor2026*`	200 con token, `roles: ["VENDEDOR"]` y sus permisos.
7	vendedor	Decodificar la parte central del token (en la consola del navegador: `atob("...")`)	Se leen `sub`, `authorities`, `iat` y `exp`; `exp - iat = 3600`.
8	vendedor	`GET /api/v1/auth/me`	200 con el usuario, sin contraseña.
9	vendedor	`GET /api/v1/productos`	200.
10	vendedor	`DELETE /api/v1/productos/1`	403 «No tiene permisos para realizar esta operación».
11	vendedor	`POST /api/v1/ventas` (venta válida)	201.
12	vendedor	`GET /api/v1/reportes/ventas-por-categoria`	403 (lo rechaza `@PreAuthorize`, no la URL).
13	almacen	`POST /api/v1/ventas`	403.
14	almacen	`PUT /api/v1/productos/{id}`	200.
15	admin	`GET /api/v1/reportes/ventas-por-categoria`	200.
16	—	Login `supervisor` / `Supervisor2026*`	200 con `roles: ["ALMACENERO", "VENDEDOR"]` y la unión de sus permisos.
17	supervisor	`POST /api/v1/ventas` y `PUT /api/v1/productos/{id}`	201 y 200: suma los permisos de sus dos roles.
18	supervisor	`GET /api/v1/reportes/ventas-por-categoria`	403: ninguno de sus roles tiene REPORTES_VER.
19	admin	`GET /api/v1/empleados` y `GET /api/v1/roles`	200; María Huamán aparece con `tieneUsuario: false`.
20	admin	`POST /api/v1/usuarios` para María (cuerpo de abajo)	201 con `roles: ["VENDEDOR"]`, sin contraseña; en la tabla, `password` empieza con `$2a$10$` y `usuario_roles` tiene la fila nueva.
21	admin	Repetir el alta con el mismo `empleadoId` y otro `username`	409 «El empleado … ya tiene un usuario» (relación 1 a 1).

N.º	Usuario / token	Petición	Resultado esperado
22	admin	Alta con `rolIds: [2, 99]`	404 «Roles no encontrados: [99]».
23	vendedor	Token con un carácter cambiado	401 «El token no es válido».
24	vendedor	Con `expiracion-minutos: 1`, esperar 2 minutos y repetir la prueba 9	401 «El token ha expirado. Inicie sesión nuevamente». Luego restaura el valor 60.

Cuerpo para crear el usuario de María (verifica los ids con `GET /empleados` y `GET /roles`; en una base nueva, María es el empleado 6 y VENDEDOR el rol 2):

```
POST /api/v1/usuarios
{
  "username": "mhuaman",
  "password": "Cajera2026",
  "empleadoId": 6,
  "rolIds": [2]
}
```

Paso 15. Versionar el trabajo

```
git add .
git commit -m "feat(seguridad): empleados, usuarios y roles con BCrypt"
git commit -am "feat(seguridad): JwtService y filtro de autenticación JWT"
git commit -am "feat(seguridad): reglas por permiso, 401/403 y login"
git commit -am "test(seguridad): pruebas de JwtService y Rol"
git push -u origin feature/seguridad-jwt
```

Haz cada commit cuando esa parte funcione y abre una solicitud de incorporación hacia `master`. Después de crear archivos nuevos usa `git add .` antes del commit. Verifica que **no** subes secretos de producción.

Producto esperado

- Rama `feature/seguridad-jwt` con al menos cuatro commits que sigan la convención y una solicitud de incorporación hacia `master`.
- PharmaBackend protegido: login JWT, contraseñas con BCrypt, permisos por rol en las reglas de `SecurityConfig` y en `@PreAuthorize`, y respuestas 401 y 403 con el formato de la API.
- Pruebas unitarias `JwtServiceTest` y `PermisosPorRolTest` en verde.
- Documento PDF `Apellido_LP2_S10_Practica.pdf` con las capturas de las 24 pruebas del paso 14.

Lista de cotejo

N.º	Criterio observable	Sí	No
1	La migración V2 crea `empleados`, `usuarios`, `roles` y `usuario_roles` con sus datos de prueba; V1 no se modificó.		
2	`Usuario` mapea la relación 1 a 1 con `Empleado` (FK única) y la muchos a muchos con `Rol` (`@JoinTable usuario_roles`).		
3	Las contraseñas se guardan con `BCryptPasswordEncoder` y ninguna respuesta devuelve la contraseña ni su hash.		

N.º	Criterio observable	Sí	No
4	El usuario recibe `ROLE_<rol>` por cada rol y la unión de los permisos de todos sus roles.		
5	`POST /api/v1/auth/login` devuelve un JWT firmado con `sub`, `authorities`, `iat` y `exp`.		
6	El secreto del JWT está en la configuración (en producción, variable de entorno) y tiene al menos 256 bits.		
7	El filtro JWT valida firma y expiración en cada petición y la API es sin estado (STATELESS).		
8	Las reglas de `SecurityConfig` exigen el permiso correcto por URL y método; los reportes usan `@PreAuthorize`.		
9	Sin token, con token vencido o alterado se responde 401 con su mensaje; sin permiso, 403.		
10	El login fallido responde siempre el mismo mensaje y no registra contraseñas en el log.		
11	Las pruebas unitarias pasan y las 24 pruebas del paso 14 tienen evidencia.		
12	La rama tiene al menos cuatro commits según la convención y una solicitud de incorporación hacia `master`.		

3. Problemas frecuentes

Síntoma	Causa probable	Solución
Todos los endpoints responden 401, incluso el login	Falta `SecurityConfig` o la ruta pública está mal escrita.	Revisa `RUTAS_PUBLICAS`: debe incluir `/api/v1/auth/login`.
Al arrancar: `WeakKeyException`	El secreto tiene menos de 256 bits o no es Base64.	Genera uno de 48 bytes en Base64 (paso 6).
Login correcto, pero siempre 401 con el token	Falta el prefijo `Bearer` o hay espacios de más al pegar el token.	El encabezado debe ser exactamente `Authorization: Bearer <token>`. En Swagger, pega solo el token.
Login siempre 401 con la contraseña correcta	La contraseña se guardó en texto plano, o el hash está incompleto.	Guarda `passwordEncoder.encode(...)` y verifica que el hash tenga 60 caracteres.
Un VENDEDOR elimina productos	La regla general está antes que la específica o falta la regla del método.	Ordena las reglas: primero `GET` con `CATALOGO_LEER`, luego el resto con `CATALOGO_ESCRIBIR`.
Acceso a reportes denegado responde 500	Falta el manejador de `AccessDeniedException` en `GlobalExceptionHandler`.	Agrega el manejador del paso 9.
La SPA muestra error de CORS después de agregar seguridad	CORS se configuró solo en Spring MVC y la verificación `OPTIONS` recibe 401.	Usa el `CorsConfigurationSource` del paso 10 y `.cors(...)` en la cadena.
«could not initialize proxy – no Session» al leer el empleado del usuario	Se accede a `usuario.getEmpleado()` (LAZY) fuera de una transacción.	Lee el empleado dentro de un método `@Transactional(readOnly = true)` del servicio, como en `AuthServiceImpl`.
La supervisora no puede editar productos	Las authorities se calculan con un solo rol.	Usa `PermisosPorRol.authorities(usuario.getRoles())`, que recorre **todos** los roles.
Flyway: «Validate failed… V1 checksum mismatch»	Se editó V1 después de aplicarla.	Deja V1 como estaba y crea los cambios en una versión nueva (V2, V3…).

4. Referencias

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. https://www.rfc-editor.org/rfc/rfc7519

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

OWASP Foundation. (s. f.). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password*Storage*Cheat_Sheet.html

Spring. (s. f.). *Spring Security reference: Servlet applications*. https://docs.spring.io/spring-security/reference/servlet/index.html

JJWT. (s. f.). *Java JWT: JSON Web Token for Java and Android*. https://github.com/jwtk/jjwt